

LiveHub

REAL-TIME BROADCASTING PLATFORM

A detailed system architecture, component relationship map, database schema, real-time message deduplication, and offline synchronization workflow specification.

PREPARED FOR:

Engineering & Architecture Review Board

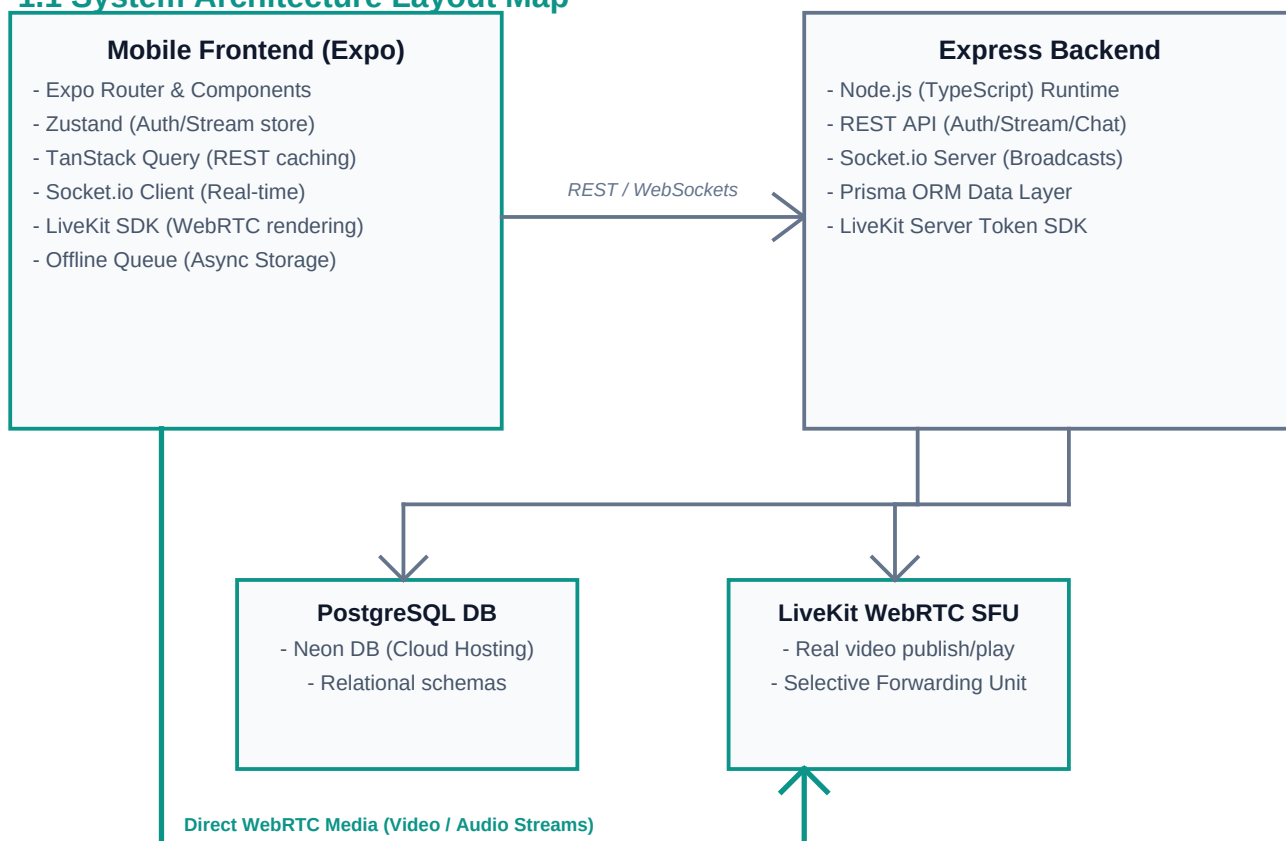
VERSION / DATE:

v1.0.0 / July 04, 2026

1. Executive Summary & Core Vision

LiveHub is a production-grade live event streaming platform built using a modern hybrid stack. It addresses the common challenges of real-time mobile publishing, interactive viewer feedback, WebRTC stream playback, and offline message queueing. The system consists of a cross-platform React Native client powered by Expo, and a robust Express backend backed by a PostgreSQL database and a LiveKit WebRTC SFU. The primary design focus is on low-latency interactions, message delivery guarantees under unstable network conditions, and high-performance layout rendering.

1.1 System Architecture Layout Map



2. Detailed Component Stack & Core Workflows

Layer	Frontend Technology	Backend Technology
Framework/Core	React Native + Expo SDK 52	Node.js with Express (TypeScript)
Routing/Views	Expo Router (File-based)	Express Endpoints / Router
State Management	Zustand State Stores	Memory Stores / In-flight request contexts
Database / ORM	AsyncStorage (local key-value)	Prisma ORM with PostgreSQL
Real-time Sync	Socket.io-client (Event handlers)	Socket.io Server (Rooms/Broadcasts)
Video/Audio Engine	LiveKit Mobile Native SDK	LiveKit Node.js Server SDK

2.1 Workflow: Stream Initialization (Creator)

1. The Creator enters a stream title and selects a category, then triggers 'Go Live'.
2. The client makes a POST request to '/streams/start' on the Express server.
3. The backend generates a unique roomName, calls LiveKit API to provision the room, and generates an access token with publisher permissions (join & publish).
4. A new Stream record is inserted into the PostgreSQL database with the status set to 'live'.
5. The backend returns the LiveKit token, room name, and database ID to the client.
6. The client updates its local Zustand 'streamStore' and navigates to the 'creator/live' screen.

2.2 Workflow: Stream Playback (Viewer)

1. The Viewer browses the feed list of active streams (fetched from GET '/streams').
2. Selecting a card navigates to '/stream/[id]' and calls the details API.
3. The backend returns a read-only LiveKit token containing only subscription rights.
4. The LiveKitRoom component initializes, joins the SFU room, and mounts the video player.

3. Real-Time Chat & Offline Synchronization

3.1 Real-Time Chat Architecture

Chat is powered by Socket.io, which connects immediately when a user enters the stream view. Upon

CORE LOGIC: MESSAGE DEDUPLICATION FLOW

- Send Action: The client assigns a UUID and deviceId to each message and adds it optimistically.
- DB Check: The backend checks if the message's uuid already exists in the Postgres database.
- Socket Sync: When the sender receives its own message broadcast back over Socket.io, the client checks the message uuid list. If found, it marks the optimistic message as synced rather than duplicating the visual entry. This ensures smooth, zero-latency chats without duplicate items.

3.2 Offline Queue & Caching Sync

When the mobile client goes offline, NetInfo detects the network change, and the useOfflineQueue hook stops making direct socket/API pushes. Instead:

1. Messages are pushed into local AsyncStorage-backed queues (OfflineQueueService).
2. The UI shows these pending messages with an 'offline' indicator icon.
3. Once NetInfo detects network recovery, the queue triggers a sync operation: it serializes all queued messages and POSTs them sequentially to the backend. The backend processes, deduplicates, stores them, and broadcasts them. The client then clears the queue.

4. Database Schema & Relational Specifications

The relational database schema is configured via Prisma ORM and deployed onto a Cloud PostgreSQL instance. The data models capture core users, live stream states, and uniquely identified chat messages.

4.1 Stream Table Schema

Field	Type	Null?	Description
id	UUID (PK)	No	Unique database identifier for the stream
creatorId	String	No	Reference to the User table for the streamer
creatorName	String	No	Cached name of the creator for fast lookups
title	String	No	Display title of the stream
roomName	String	No	Unique LiveKit WebRTC room name
status	String	No	Active status: 'live' or 'ended'
viewerCount	Integer	No	Count of active concurrent viewers
category	String	No	Category filter (e.g. Gaming, Tech, Music)
startedAt	DateTime	No	Timestamp of when the stream was created
endedAt	DateTime	Yes	Timestamp of when the stream was ended

4.2 ChatMessage Table Schema

Field	Type	Null?	Description
id	UUID (PK)	No	Unique auto-incrementing database message ID
uuid	String	No	Unique client-generated message ID (for deduplication)
deviceId	String	Yes	ID of sender's device (for security audits)
streamId	String	No	Reference to the associated Stream record
senderId	String	No	User ID of the message sender
senderName	String	No	Cached display name of the message sender
message	Text	No	Content text of the chat message
createdAt	DateTime	No	Timestamp of when the message was sent